

目录

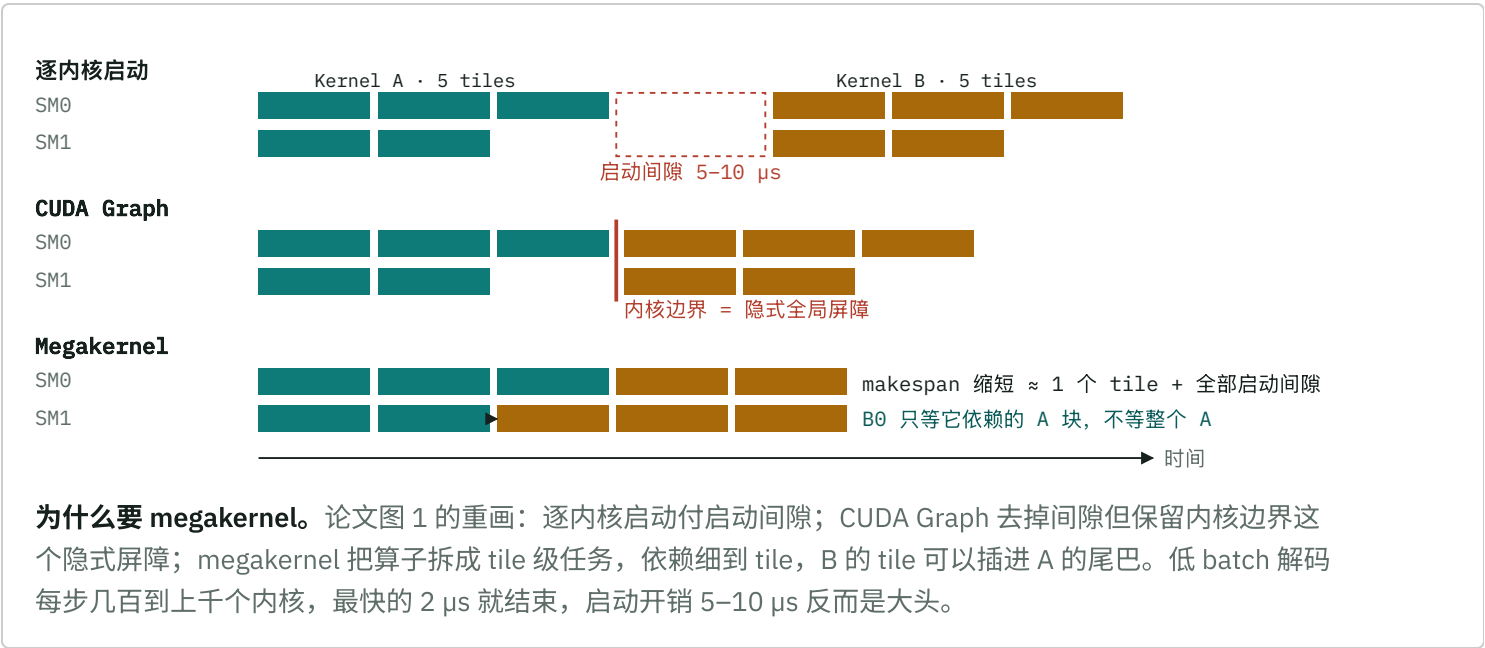
- 01 论文拆解：ETC 到底做了什么
- 02 从论文读出的七个架构判断
- 03 ETX：目标、非目标、三个硬决定
- 04 分层总览
- 05 性能模型：两个下界与四个杠杆
- 06 L1 · Tile 契约（DSL 无关怎么落地）
- 07 L2 · Event Tensor IR
- 08 L3 · 机器模型（chiplet 感知）
- 09 L4 · 放置与调度 pass
- 10 L5 · 后端 lowering 与运行时 ABI
- 11 动态负载：三个层次
- 12 负载覆盖矩阵
- 13 验证策略
- 14 与 ETC / MPK / Hazy 的差异
- 15 里程碑、风险、未验证清单
- 16 来源

ETX：一个 DSL 无关、chiplet 感知的动态 Megakernel 编译器

把「多个 GPU 算子融合进一个持久化内核，同时处理动态 shape 与数据依赖」做成一个编译器，而不是一份手写内核。抽象沿用 Event Tensor（MLSys 2026），再补上论文没有的两样东西：显式的机器模型，和按代价选出来的调度策略。

依据 [arXiv 2604.13327 v2](#) (Event Tensor / ETC, CMU + NVIDIA)

参照 MPK、Hazy no-bubbles、fleet-mi300x (MI300X 上的手写 megakernel) 状态 设计稿，未实现



01 · 论文拆解

ETC 到底做了什么

论文的核心是一个 IR 级别的抽象，不是一个新调度器。它把 megakernel 里「谁等谁」这件事，从散落在手写内核里的信号量，提升成编译器里的一等张量。剩下的静态 / 动态调度，都是对这个张量做变换。

1.1 三个语言构件

DEVICE FUNCTION

任务网格

一个函数，按多维坐标启动成一组 tile 级任务。每个任务在一个 SM 上跑，内部可以 warp 特化、用 tensor core。任务坐标是符号表达式，比如 $(n, 4)$ 。

EVENT TENSOR

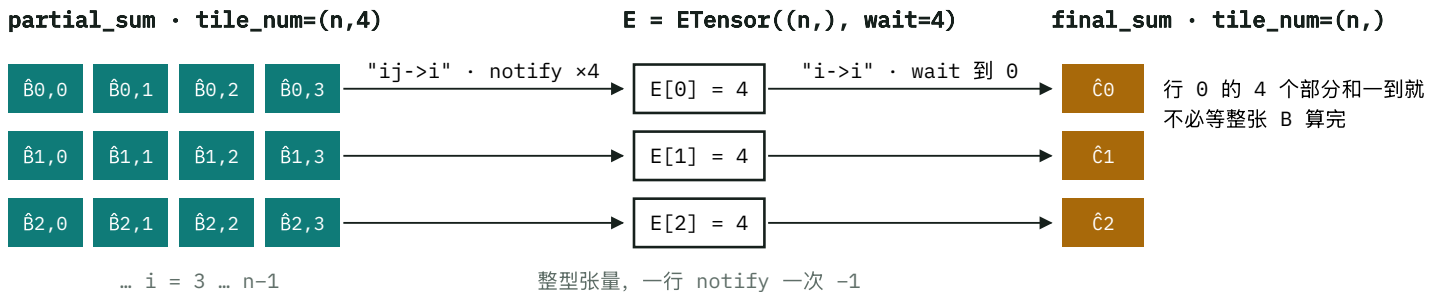
事件张量

多维数组，每个元素是一个事件：初始等待计数 `wait_count`，支持 `notify()`（原子减一）、`wait()`（自旋到零）。动态调度下事件归零还能触发消费者任务。shape 可以含符号维度。

GRAPH FUNCTION

图函数

由 `call_device` 组成的计算图，节点既有数据张量也有事件张量。每次调用用 `in_edges` / `out_edges` 加一个 einsum 风格映射（如 `"ij->i"`）声明 task 坐标到 event 坐标的关系。



`n` 是符号变量：编译一次，运行时用具体 `n` 实例化坐标空间。

Event Tensor 的最小例子（论文图 3）。split-K 求和的第二阶段第 i 行只依赖第一阶段第 i 行的 4 个 tile。逐内核执行时第二阶段要等第一阶段全部结束；有了 `E[i]`，行与行之间是流水的。一张张量就表示了成千上万个事件，运行时不需要物化任务图。

1.2 两种动态性怎么进 IR

- shape 动态 Tensor 与 ETensor 的维度都可以是符号变量（batch、seq len）。编译产物是一个模板，运行时填具体值就得到当次的任务图，不重编译、不重新捕获 graph。这是它能 AOT 的根本原因。
- 数据依赖动态 两个机制。**数据依赖更新**：谁通知哪个事件由运行时张量决定，如 `"i->topk[i,:]"`（token i 通知它被路由到的 k 个 expert 事件），并且事件的初始计数也在算 topk 的那个 tile 里一起算出来。**数据依赖触发**：一个事件触发多少消费者由运行时决定，如 `"i->range(indptr[i], indptr[i+1])"`（expert i 触发它需要的 GroupGEMM tile 数）。
- 不变量 整条依赖链仍然是前馈的：Attention → TopK → 分组 → GroupGEMM。动态只出现在后段的通知目标和触发范围上，图本身没有环、没有任意控制流。

1.3 两条调度变换

维度	静态调度 (§3.1)	动态调度 (§3.2)
谁决定任务顺序	host 在启动前为每个 SM 排好队列（论文用 round-robin），物化进全局内存	片上调度器：事件归零时把消费者任务原子 push 进就绪队列，空闲 SM 原子 pop
依赖如何保证	消费者任务开头 <code>wait()</code> ，生产者任务结尾 <code>notify()</code>	同样的 <code>notify / wait</code> ，外加 <code>push / pop</code>
shape 动态	采样一组代表 shape 预排队列；没见过的 shape 复用「下一个更大的」采样值的队列	天然支持，顺序在运行时定
数据依赖动态	保守退化：相关 <code>notify / wait</code> 改写成 <code>E[0]</code> ，等于回到全局屏障	天然支持
开销	最低，只有原子与自旋	队列 <code>push / pop</code> ；中心化队列在全局内存，作者承认规模上有争用；附录 E 用 early push（生产者被派发即 push 消费者）把 push 挪出关键路径
适用	可预测负载、延迟敏感的小 batch、跨卡场景	不规则负载（MoE 路由）、通信抖动

两者 lowering 之后都只剩两块运行时状态：一张整型事件张量（复用现有张量与内存规划）和调度器的任务队列。没有任务图遍历器，没有解释执行层。编译流水线是：图级优化 → tile 级优化 → 静态或动态调度变换 → 预取改写 → 持久化内核代码生成 → 静态队列物化。

1.4 数字

实验（8×B200，NVLINK）	对手	结果
GEMM + Reduce-Scatter （TP=8，8192 token）	cuBLAS+NCCL、 TP-Async、 Triton- Distributed、 cuBLASMp	最高 1.40×（对未融合）
All-Gather + GEMM	同上	最高 1.40×
MoE 层，Qwen3-30B-A3B （128 expert，top-8）	Triton 3.4、 FlashInfer 0.2.14	最高 1.23×（1024 token）
端到端 TPOT，30B-A3B， bs=1	vLLM 0.11、 SGLang 0.5.3 （都开 CUDA Graph + torch.compile）	1.48× / 1.20×
端到端 TPOT，Qwen3- 32B，TP=1	同上	最高 1.15× / 1.09×
端到端 TPOT，Qwen3- 32B，TP=4	同上	0.99–1.06×，SGLang 反而更快（CPU 侧调度更轻）
Qwen3-32B 引擎预热	SGLang 583 s / vLLM 123 s （JIT + 51 / 67 次 graph 捕获）	35 s，0 次捕获；离线编译 107 s
MoE 层：static vs dynamic （对同代码的未融合 megakernel）	1 / 128 / 1024 / 4096 token	static 1.03 / 1.02 / 1.04 / 1.02；dynamic 0.95 / 1.06 / 1.07
Qwen3-32B TP=4：static vs dynamic	bs 1 / 16 / 32 / 128	static 1.09 / 1.06 / 1.07 / 1.06； dynamic 0.83 / 0.82 / 0.81

高亮两行是本设计最重要的证据：同一套原语，调度策略选错就从 +9% 变成 -17%。所以调度策略不能由用户凭直觉配，得由代价模型按负载与拓扑选。

从论文读出的七个架构判断

1. **事件就是整型计数器，这个决定要保留。**它让运行时状态压缩到两块内存，让符号 shape 的编译器基础设施能直接复用。任何「事件对象」「任务图节点」都是退步。
2. **调度是编译器变换，不是运行时组件。**static / dynamic 都是对同一个 IR 的改写，运行时只剩 push / pop 两个函数。这使得策略可以按子图组合，而不是全局二选一。ETX 的 hybrid 调度就是在这个基础上做的。
3. **机器模型是隐式的，而且只在 B200 上成立。**论文假设 SM 均质、L2 统一相干、有 multimem 归约写、有 NVLink。一张 MI300X 至少四处不成立：L2 是 8 段各 4 MB 的 XCD 私有缓存；没有 multimem；跨 XCD 原子走 Infinity Fabric；系统域原子对内存类型有要求。论文说抽象「不针对特定 GPU 代际」，是对 IR 说的，实现没验证过。
4. **静态调度对动态性的兜底太粗。**shape 用「下一个更大的采样值」的队列；数据依赖直接退化成 `E[0]` 全局屏障。这在 MoE + 低 batch + 跨卡这种最常见的组合上正好是最差的一档，需要 hybrid 补上。
5. **动态调度的代价是拓扑函数。**TP=4 上 0.83× 说明 push 一旦跨设备就得不偿失；中心化全局队列作者也承认有争用。队列必须分层，push 必须有域界。
6. **tile 质量与融合收益必须解耦。**作者承认生成的 GEMM tile 在某些配置下不如 cuBLAS，导致 TP=4 上偶尔落后。系统必须允许内联手写 tile（CUTLASS / CK 风格），并且评测时把「融合的收益」和「tile 的质量」分开报。
7. **每步一次启动，图内没有循环。**论文的 megakernel 覆盖一次解码迭代，continuous batching 的形状变化靠符号 shape 吸收，但内核本身每步重启。跨步持久化（内核常驻、host 通过环形缓冲喂步描述）论文没做，这是延迟敏感场景下另一个可拿的量。
8. **只有任务级的依赖变换，没有「改图」的变换。**ETC 能把一个算子拆成 tile 并细化依赖，但不会删相位：没有以重算换事件、没有跨屏障的权重预取。小 batch 下关键路径由相位数和每相位固定延迟决定，这两项正是手写内核（Hazy、fleet-mi300x）和它拉开差距的地方。第 05 节把这笔账算清楚。

2.1 2026 年的生态：十条设计轴，以及 ETX 在每条轴上的取值

论文之外，一年内出现了十几个 megakernel 系统。把它们摆在一起，差异集中在十条轴上。ETX 的每个取值都能在这张表里找到先例或反例。

设计轴	已有系统的取值	ETX 的取值与理由
谁拥有调度器	专用调度 SM (MPK 4 个 SM; Fleet 每 XCD 一个 workgroup)· 每个 SM 自取全局计数器 (Hazy 70B、Cohere)· host 预排、内核内无调度器 (Hazy 1B、Ada-MK、Kog、ForgeMegaKernel; MPK 的 2026 路线图也转向全静态)	按子图选: 静态段无调度器; 动态段每 domain 一个本地队列, worker 自取; 不设专用调度 SM (fleet-mi300x 上它只做事件镜像, 带宽受限时占 2.6% CU 无所谓, 计算受限时不划算)
队列拓扑	每 worker 队列 (MPK JIT/AOT 双队列、Fleet)· 单全局队列 (Hazy 70B)· 按算子的 drain 队列 (Cohere 的 <code>ATTN_DRAIN</code> / <code>MOE_*_DRAIN</code>)· 无队列 (Kog、Ada-MK)	domain 本地队列 + 每 worker 静态表; 数据依赖算子可再按算子拆 drain 队列 (Cohere 的做法与 domain 分层正交, 可叠加)
依赖何时解析	JIT (事件触发后才派发) vs AOT (预入队、worker 自旋): MPK 两者混用, Cohere / Hazy 全 AOT 自旋, Ada-MK 编译期消灭	静态段 AOT 自旋 (一跳); 动态段 JIT (两跳); ETC 的 early push 作为动态段默认
事件粒度与编码	带扇入扇出范围的事件对象 (MPK)· 每边一个计数器 (Hazy、Forge)· 每任务一个目标计数 (Cohere)· 数据缓冲里的哨兵值 (Kog, 0.8 μ s vs 原子 7.6 μ s)· 带 epoch 的标志 (mKernel)	事件张量 (整型) + epoch 戳: 跨步常驻时不必每步清零, 比较 <code>count == epoch_target</code> 即可; 哨兵值作为可选 lowering (能力位 <code>sentinel_signal</code>)
信号的内存模型	GPU-scope 原子 + threadfence (NVIDIA 系)· chiplet 分层: L2 本地计数 + 每 XCD 一次 <code>sc0 sc1 fence</code> (Fleet, 跨 chiplet 信号减少 14.5 $\times$)· 编译器可见的 acquire/release token (Triton-distributed 的 <code>consume_token</code> , Iris 的 block/gpu/sys scope)	scope 进 IR, lowering 表按架构给 fence; last-arriver 一次回写 (Fleet 与 fleet-mi300x 都验证过)
动态 shape	按 batch 特化多张图 (MPK、Fleet、ETC-static)· 编译期固定 (Forge、Hazy 1B、Ada-MK)· 只对数据依赖算子用工作窃取 (Cohere、MPK 的 JIT attention)· 角色自适应 (mKernel)· 确定性填充 (MoK)	符号 shape 模板 + shape 桶; 数据依赖段本地队列; 不做 padding 兜底
共享内存 / LDS 归属	页分配器、生命周期跟随指令 (Hazy 13 $\times$ 16 KB 页、Forge、Ada-MK 的页状态机)· 按 task 类型最坏分配 (MPK、Fleet, 占用率抱怨的来源)	L5 骨架内置 LDS 页分配器: 下一 tile 的权重装载与当前 tile 的写回重叠; 页数由资源类与 LDS 容量 (64 / 160 / 228 KB) 算出
codegen 路径	超优化器 + 转译 (MPK)· MLIR (Ada-MK、cuTile)· 手写 + tile 库 (Hazy、Cohere、MoK、Kog)· agent 合成 + 校验器 (Forge、AutoMegaKernel)· Triton + SHMEM 原语 (Triton-distributed、Iris、DITRON)	不选边: link 模式吃手写与厂商库, host-DSL 模式吃 Triton; 调度骨架由 ETX 生成
硬件覆盖	NVIDIA 占绝大多数; AMD 只有 Fleet (MI350X, MPK 衍生)、Kog (MI300X, 专用)、Iris / DITRON (通信原语); Rubin 宣称 tile 级依赖触发由硬件接管	chiplet 是一等概念; <code>wait</code> 的 lowering 允许落到硬件触发 (能力位 <code>hw_tile_trigger</code>), 硬件接管时 ETX 只少生成一段自旋
融合范围	整个解码步 (MPK、Hazy、Cohere、Forge、Kog)· 单层类型 (FlashInfer MegaMoE、MegaGDN、MoK)· 服务循环本身 (Blink、MPK 的内核内准入)	子图级: 融合失败的部分回退逐内核; 跨步常驻是可选模式

两条值得单独记住的证据: Cohere 在 H100 上用「host 预排静态波次 + 按算子的 drain 队列」拿到 30B-A3B bs=1 对 vLLM 1.58 $\times$, 说明 hybrid 是对的方向; AMD 自己的 Fleet 在 MI350X 上用「XCD 内 L2 本地计数 + 每

XCD 一次 GPU-scope fence」比直接移植的 MPK 快 1.13–1.30×，说明 chiplet 感知不是可选项。

从 FLEET-MI300X 带来的经验

我们在 MI300X 上手写过同类 megakernel (DeepSeek-V2-Lite bs=1 解码, 3.60 ms/token, 比同机 vLLM 快 20%)。那份工作证明了三件事：跨 XCD 事件与 XCD 内事件的代价差一个量级，一层 MoE 里能把全局事件从 10 压到 3、XCD 局部事件 24 个；生产者 `buffer_wb12` 回写不能省，消费者 L1 失效不能省；持久化内核的 workgroup 到 XCD 映射不是契约，要在启动时探测。这三条都直接进了 L3 与 L5 的设计。

ETX：目标、非目标、三个硬决定

ETX（Event Tensor eXtended）是一个位于 tile DSL 之上的编译器：它不生成 tile 内的指令，只负责切 tile、连依赖、排调度、放置到机器上、生成持久化内核的骨架，再把各家 DSL 编译出的 tile 体链接进去。

目标 · 按优先级

- 1. DSL 无关：Triton、TileLang、CuTe DSL、CK、手写 HIP / CUDA 都能接。
- 2. 硬件可移植：同一份 IR 在 CDNA2 / CDNA3 / CDNA4、RDNA、NVIDIA Hopper / Blackwell 上都出正确且合理的代码，chiplet 是一等概念。
- 3. 动态负载一等支持：shape 动态、数据依赖动态、跨步动态（continuous batching、speculative decoding）。
- 4. 调度决策自动化且可解释：编译器按代价表选 static / dynamic / hybrid，并打印理由。
- 5. 可验证：每个融合内核都有可逐元素比对的未融合参考，死锁能定位。

非目标 · 明确不做

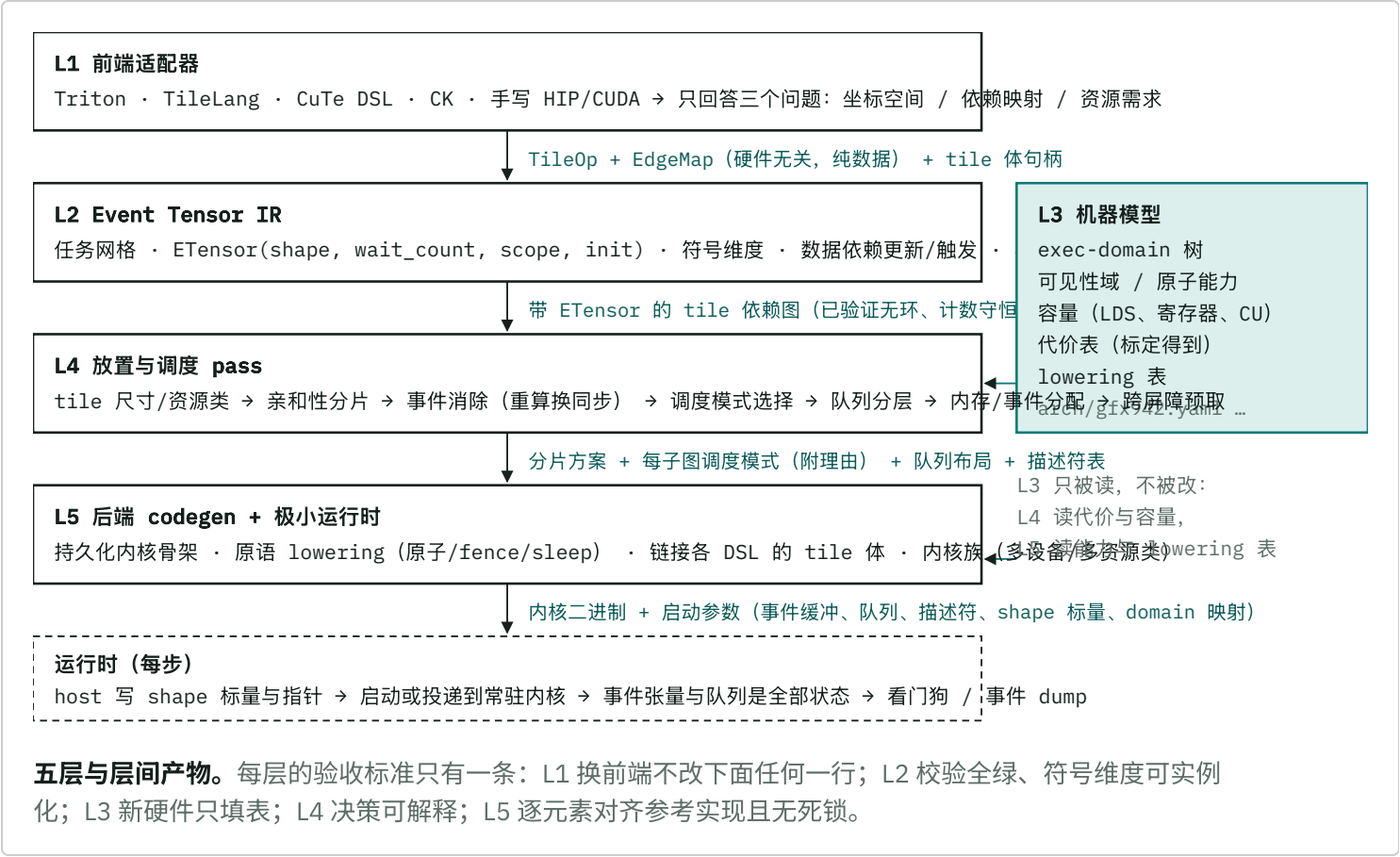
- 不做训练框架；第一版只做推理侧融合。
- 不自研全部 tile 级代码生成；允许内联已调优的手写 tile。
- 第一版不做跨节点；IR 不写死单机，调度器先看单机拓扑。
- 不追求所有算子都能融合；融合失败要优雅退化成部分子图融合 + 逐内核执行。
- 不处理任意控制流；依赖必须能写成仿射映射或运行时范围映射。

三个硬决定

决定	内容	违反的代价
硬件差异只以数据存在	pass 里不许出现 <code>if arch == gfx942</code> 。所有差异写在 <code>arch/*.yaml</code> 的能力表、代价表和 lowering 表里。	新硬件接入要改 pass，每个 pass 都变成 if 森林
能力与代价分开建模	capability 是布尔 / 枚举，决定「能不能做」，错了是错的代码；cost 是数值，决定「值不值得做」，错了只是慢。	混在一起后没法回答「这个优化在这台机器上是不支持还是不划算」
调度策略由代价模型选	每个子图独立决策 static / dynamic / hybrid，输入是负载特征 + 拓扑代价，输出附带理由。	论文 TP=4 上 0.83× 的重演

再加一条贯穿全部层的纪律：**依赖语义、机器事实、放置决策三条线永不混写**。依赖错了结果错或死锁，放在 L2；机器事实错了是静默错或性能差，放在 L3；决策错了只是慢，放在 L4。每条线出问题都能只查一层。

分层总览



两个下界与四个杠杆

在写任何 pass 之前先把账算清楚：megakernel 到底能从哪里拿时间。解码一步的时间有两个下界，取大者：

$$T_{\text{step}} \geq \max\left(\frac{\sum \text{bytes}}{BW_{\text{HBM}}}, \sum_{\text{相位}} \left(\frac{\text{bytes}_{\text{相位}}}{BW_{\text{HBM}}} + T_{\text{fixed}}(\text{相位})\right)\right)$$

带宽项
关键路径项

$$T_{\text{fixed}}(\text{相位}) = \text{信号延迟} + \text{排空 (最慢 worker 与平均的差)} + \text{下一相位的 prologue 爬坡}$$

大 batch 时带宽项占优，融合能拿的是相位边缘的空洞（wave 量化、启动间隙），收益 5% 到 15%。小 batch 时关键路径项占优，这时 megakernel 不是「重叠」而是「把链弄短」。同一套技术在两种场景下的倍数差一个量级，原因全在基线离下界多远：

场景	字节 / TOKEN	带宽下界	基线	MEGAKERNEL	读法
Llama-1B, H100 (Hazy)	≈ 2.4 GB	≈ 0.7 ms	vLLM ≈ 2.5 ms (28% MBU)	< 1 ms (78% MBU)	模型极小，基线被几百次 launch 淹没，所以 2.5×
Qwen3-32B, B200 (ETC)	≈ 66 GB	≈ 8.2 ms	vLLM ≈ 14 ms (57%)	≈ 12.5 ms (65%)	基线已在下界六成，理论最多 1.77×，拿到 1.15×
DeepSeek-V2-Lite MoE, MI300X (fleet-mi300x)	4.9 GB	0.93 ms	vLLM 4.52 ms (21%)	3.60 ms (26%)	差距不是 launch，是 27 层 × 7 个相位的串行固定延迟

一层 MoE 的 133 μs 去哪了 (fleet-mi300x 实测 + 估计拆分)

带宽项 31 μs

prologue 搬运 / 折叠 ≈ 36 μs

信号 + 排空 ≈ 20 μs

只有少数 worker 在跑的串行小任务 ≈ 46 μs

7 个相位 × 3 个全局事件 + 24 个 XCD 局部事件；带宽项 = 166 MB / 5.3 TB/s。

后三段是关键路径项，都不搬字节。它们分别对应下面的杠杆 2、2 和 1；杠杆 3 用下一相位的字节把它们填满。

拆分比例是估计值，只有总数和 prologue 是实测；阶段 0 的标定要把它换成每相位的 trace。

关键路径项是主体。26% 的 MBU 不是 tile 慢，隔离出来的 GEMV 已到可达带宽的 93%；是每层有 100 μs 的时间里内核在跑却几乎不搬字节。

四个杠杆，各吃一段

杠杆	吃掉哪一段	做法	落在哪个 PASS	下限
1·链变短	不必要的相位边界	以重算换事件：小生产者内联进消费者的 prologue，事件和任务一起删（Pass 3）	L4 Pass 3	只剩天然的全量依赖点：一层 MoE 是 3 个（o_proj 要全部 head、router 要完整向量、下一层要全部 expert 的和）
2·每一环变便宜	信号延迟、prologue	XCD 局部事件、哨兵值代替计数器（0.8 vs 7.6 μ s）、last-arriver 回写；prologue 的搬运在 tile 设计里消掉（每 XCD 一次而不是每 worker 一次）	L5 lowering 表；L1 tile 契约	信号 $\approx$ 0.2 到 0.8 μ s / 环
3·跨屏障重叠	等事件的空转	权重不依赖任何事件：worker 在自旋时把下一相位自己那份权重流进 LDS / 寄存器，事件一到直接算（Hazy 到 78% MBU 的主要办法）	L4 Pass 7 + LDS 分页 + 资源类	固定延迟被下一相位的字节填满
4·细粒度事件	排空（等最慢的 worker）	全局事件拆成按 head / 按行的事件（论文的 <code>ij->i</code> ），消费者只等自己的生产者，straggler 只拖累自己的下游	L2 边映射 + Pass 2 分片	真全量的依赖点仍要等全部

合起来，一层的时间从「 $7 \times (\text{字节/带宽} + \text{固定})$ 」变成「 $3 \times \text{字节/带宽} + 3 \times \text{被预取填掉的固定延迟}$ 」，才有机会贴近 31 μ s 的带宽项。这四个杠杆里，ETC 只有第 4 个；MPK 和 Fleet 有 2 的一部分；Hazy 手写了 1、2、3。ETX 把四个都做成 pass，这是它对手写内核的追赶路线，也是它对 ETC 的主要增量。

对收益的诚实预期

30B 以上 dense 模型、 $bs \geq 16$ ：对 vLLM / SGLang 10% 到 30%，上限由基线 MBU 决定。MoE 小 batch 与 TP：基线 MBU 只有 10% 到 25%， $1.5\times$ 到 $3\times$ 是有物理空间的，但要同时拿到杠杆 1 到 3，只把算子塞进一个内核拿不到。评测一律报 MBU 和每相位的 trace，不单报倍数。

Tile 契约：DSL 无关怎么落地

megakernel 编译器只做三件事：切 tile、连依赖、排调度。三件事各需要一份输入。所以任何前端只要能回答三个问题，就能接入；答不出来的，就不该硬接，而应退化到逐内核执行。

问题	答案的形式	说明
tile 坐标空间是什么	<code>Grid = (expr_0, expr_1, ...)</code>	允许符号表达式，如 <code>(B*H, ceil(S/128))</code> 。
每个 tile 读写什么	显式： <code>coord → {(tensor_slice, producer_coord)}</code> 声明式： <code>reads/writes</code> 的切片表达式，由 ETX 反推	手写 IR 给显式；Triton 类前端给声明式。反推失败（切片不是仿射）就报错，不猜。
资源需求	<code>Resource(threads, lds_bytes, vgpr, agpr, tensor_core, cluster?)</code>	抽象需求；映射到 64 KB 还是 160 KB LDS 是 L4 的事。

5.1 两种接入模式（这是 DSL 无关的真正机制）

「DSL 无关」不能只是一句 IR 层的声明。tile 体最终要和持久化循环编到同一个内核里，不同 DSL 能给出的产物形态不同，所以后端要提供两条路：

LINK 模式

tile 体是可链接的设备函数

前端把每个 tile 体编成一个符合固定 ABI 的设备函数（`-fgpu-rdc / -rdc=true`），ETX 用 HIP / CUDA C++ 生成持久化循环与 dispatch switch，链接时合并。CUTLASS、CK、CuTe DSL（导出 device function）、手写内核走这条路。

- 优点 tile 体不用知道 ETX 存在；可以内联厂商库级别的 tile。
- 代价 跨翻译单元内联受限，寄存器分配按全内核取并集。

HOST-DSL 模式

持久化循环用 tile 的 DSL 生成

Triton 不导出可链接的设备函数，但 `@triton.jit` 函数可以互相调用。ETX 为这类前端生成一个同 DSL 的持久化内核：循环、pop、wait、dispatch 都用该 DSL 写，运行时原语通过 `tl.atomic_*`、extern 库函数或 `inline_asm` 提供（Triton-Distributed 已证明这条路可行）。

- 优点 整内核在一个编译单元里，能整体优化。
- 代价 每支持一个 DSL 要写一个骨架模板；DSL 的表达力限制骨架（比如 Triton 没有 warp 特化）。

5.2 Tile ABI（link 模式）

```
// 每个 tile 体的固定签名；前端负责按它导出
struct etx_ctx {
    int32_t    coord[4];          // 任务坐标（符号 shape 已实例化）
    const int32_t* shape;         // 本步的符号标量表（B, S, n_expert_tiles, ...）
    void* const* args;           // 张量指针表，索引由 L4 分配
    etx_event*  events;          // 事件缓冲基址（整型）
    uint32_t    domain, worker;   // 所在 exec domain 与 worker 编号（L5 启动时探测）
    void*       lds;              // 本 worker 的 LDS 基址，大小按资源类
};

extern "C" __device__ void etx_tile_<name>(const etx_ctx*);

// tile 体内可选调用的运行时原语（有默认实现，也允许前端重写）
__device__ void    etx_wait  (etx_event*, int scope);
__device__ int32_t etx_arrive(etx_event*, int scope); // 返回剩余计数
__device__ void    etx_push  (etx_queue*, etx_task*);
__device__ bool    etx_pop   (etx_queue*, etx_task*);
__device__ void    etx_backoff(int level);           // s_sleep / nanosleep
```

ABI 有意只暴露六样东西：坐标、标量、指针表、事件、位置、LDS。任何需要更多信息的 tile 体，说明它试图做调度器的事，应该退回 L4。

5.3 前端矩阵

前端	模式	依赖给法	备注
Triton（NVIDIA / AMD 后端）	host-DSL	声明式（切片反推）	无 warp 特化、无 TMA 多播控制；适合 GEMV、norm、gather/scatter 类 tile。
TileLang / TVM TensorIR	两者皆可	声明式	论文的实现路径；已有符号 shape 支持。
CuTe DSL / CUTLASS	link	显式	GEMM、attention 主力 tile；Hopper / Blackwell 特性全开。
Composable Kernel（AMD）	link	显式	AMD 侧 GEMM / attention 主力；MFMA、LDS 布局都在 CK 内部解决。
手写 HIP / CUDA	link	显式	fleet-mi300x 的 GEMV 与 MLA tile 就是这一类。

Event Tensor IR

6.1 核心类型

```
Tensor = (shape[sym|const], dtype, layout_hint)
ETensor = (shape[sym|const], wait_count[sym|const|runtime], scope, domain_id, init_kind)
TaskGrid = (grid[sym], tile_ref, resource_class, in_edges{ETensor: map}, out_edges{ETensor: map},
            pure: bool, out_to_in: map) # pure + 输出坐标→输入切片映射, Pass 3 内
# 联重算的前提
Step = (graph_fn, shape_scalars, arg_table) # 一次调用 = 一次迭代
```

- `wait_count` 归零前需要的 notify 次数。可以是常量、符号（`WORLD_SIZE`），或运行时值（路由到 expert i 的 token 数），运行时值必须由某个上游 tile 写出。
- `scope` 事件的可见性范围：`workgroup < cluster < domain < device < system`。`domain` 是 ETX 相对论文新增的一级，对应 XCD / GCD / die。论文的事件只有一种 scope（device），这在均质 L2 上没问题，在 chiplet 上是一个量级的代价差。
- `domain_id` 这个事件主要服务哪个 exec domain 的任务。由 L4 的分片 pass 填，L2 里留空。
- `init_kind` `static`（常量）、`per_step`（host 每步写入，如 batch 相关计数）、`runtime`（某个 tile 写入，如 MoE 路由计数）。

6.2 边映射的四种形式

形式	例子	用途	编译期能分析什么
仿射 (einsum 子集)	"ij->i"	规约、拆分、静态依赖	精确扇入扇出，可静态排队
索引算术	"ij->(i/2)j"	tile 尺寸不匹配（RS tile 是 MM tile 两倍）	同上
运行时范围	"i->range(indptr[i], indptr[i+1])"	一个 producer 触发变数量消费者（MoE expert → GroupGEMM tile）	只知道上界；静态调度只能按上界排队并用 wait_count 兜底
运行时 gather	"i->topk[i,:]"	producer 通知哪些事件由数据决定	只知道扇出 k；计数由 init_kind=runtime 的 tile 初始化

约束是分析能力的前提：映射必须是仿射的，或 `range`(仿射, 仿射)、`gather`(张量) 形式；禁止任意控制流进边映射；运行时值张量必须来自上游 tile 的写出，且带 release 语义。

6.3 语义（这是 IR 的一部分，不是后端细节）

```
arrive(ev)  ≡  release-fence; atomic_dec(ev.count)           →  结果为 0 时 ev 触发
wait(ev)    ≡  spin_until(ev.count == 0); acquire-fence
trigger(ev) ≡  arrive 返回 0 时, 把 ev 的消费者任务集合 push 进调度器 (仅动态 / hybrid)
```

生产者的写必须在 arrive 之前对消费者可见 (release)，消费者的读必须在 wait 返回之后 (acquire)，而且是在事件的 **scope** 上成立。后端若发现目标硬件在该 scope 上无法提供这个语义 (见 L3 的原子陷阱)，必须编译报错，不许静默降级。

6.4 校验 pass (必须全绿才允许进 L4)

校验	检查	不做的后果
无环	任务级依赖图无环 (符号维度取任意值都成立)	死锁，现场极难定位
计数守恒	<code>wait_count</code> 等于实际扇入 (运行时范围取上界，且初始化 tile 必须在图中)	早触发 (结果错) 或永不触发 (死锁)
运行时值先行	<code>indptr</code> / <code>topk</code> 类张量的写出 tile 在依赖图上严格早于所有读它的 arrive / trigger	读到旧路由，MoE 结果错但看起来正常
tile 覆盖唯一	每个输出元素恰好被一个 tile 写	静默数据竞争
scope 单调	跨 domain 的边所用事件 $\text{scope} \geq \text{domain}$ ；跨设备 $\geq \text{system}$	本地可见、远端看不见
协同驻留可满足	持久化内核的 worker 数 $\leq$ 目标的共驻上限 (按资源类算)	worker 不共驻 → 等一个永远不会被调度的生产者

机器模型：让 chiplet 成为数据

论文的 IR 是硬件无关的，但它的实现与全部评测只在一种机器上。要扩展到不同硬件，不是给后端加分支，而是把「一台机器长什么样」写成一棵树、几张表，让 L4 与 L5 只读表。

7.1 exec-domain 树

MI300X · gfx942

device (1 个 HIP 设备)

XCD0 XCD1 ... XCD7

每 XCD 38 CU · L2 4 MB 私有
跨 XCD 只有 256 MB Infinity Cache
LDS 64 KB · wave64 · 5.3 TB/s
domain = XCD, 8 个

MI250X · gfx90a

device 0 device 1

GCD0 · 110 CU GCD1 · 110 CU

两个 GCD 是两个 HIP 设备
每个 8 MB / GCD · 无 Infinity Cache
浮点原子不能过 Infinity Fabric
一卡 = 两个内核实例

B200 · sm_100

device (2 die, 软件看 1 颗)

SM ×148 · cluster 可选 · L2 统一

DSMEM / TMA 多播 / multitem
mem 228 KB / SM · wave32
die 间有延迟差, 但相干
domain = 整卡 (可选 die 亲和)

MI355X · gfx950

device

XCD0 XCD1 ... XCD7

归约每 XCD 32 CU · L2 4 MB 私有
LDS 160 KB · 8 TB/s HBM3E
树与 MI300X 同形, 容量不同
同一份 yaml 改数字

ETX 只认这棵树。同一 domain 内：事件走本地 L2, 代价 T_local；跨 domain：走共享缓存 / 互联, 代价 T_cross；跨 device：走 P2P 新硬件 = 新的一棵树 + 三个代价系数 + 一张能力表。RDNA (WGP、wave32、无 MALL 分片) 同样只是树的一种形状。

四种机器的 exec-domain 树。 chiplet 不是特殊情况，是树多一层。MI250X 的两个 GCD 是两个设备，所以「一张卡」在 ETX 里是两个内核实例共享 system-scope 事件；这与多卡 TP 是同一条代码路径。

7.2 能力表（决定正确性）

```

# arch/gfx942.yaml (节选)
name: gfx942          family: CDNA3          wave_size: 64
exec_domains:
  - {name: xcd, count: 8, cus: 38, l2_mb: 4}
shared_cache: {kind: infinity_cache, mb: 256, coherent: false}  # 内存侧，不参与相干
resources: {lds_kb: 64, regs_per_lane: 512, max_waves_per_simd: 8}
visibility:          # 哪个 scope 需要什么 fence / 轮询方式 / 内存类型 (来源: LLVM gfx
x942 memory model)
  domain: {release: "s_waitcnt vmcnt(0)",          acquire: "buffer_inv sc0",          poll: "load
sc1", memory: any}  # L1 写穿，同 L2 内只需失效 L1
  device: {release: "buffer_wb12 sc1; s_waitcnt", acquire: "buffer_inv sc1",          poll: "load
sc1", memory: any}  # 回写本 XCD 的脏 L2 行
  system: {release: "buffer_wb12 sc0 sc1",          acquire: "buffer_inv sc0 sc1", poll: "load
sc0 sc1", memory: fine_grained}  # coarse 会被静默降级到 device
capabilities:
  cluster_launch: false          # gfx942 无; gfx1250 才有 cluster scope, 不支持时 LLVM 退化为
agent scope
  dsmem: false
  multicast_reduce: false          # 无 multimem 对应物，也没有交换机 (8 卡是点对点 mesh)
  async_copy_to_lds: dword_only  # 无 TMA / cp.async; gfx950 放宽到 128 bit/lane
  fp_atomic_over_fabric: true     # 所有 RMW 原子都转发到 Infinity Fabric; MI200 浮点原子为 false
  wg_to_domain_map: discover      # 文档只说 round-robin; 有公开记录测到 (k+4) mod 8, 所以启动时
读 HW_ID.XCC_ID
  cooperative_launch: true        # 超过共驻上限返回 hipErrorCooperativeLaunchTooLarge, 而不是
死锁
  sentinel_signal: true           # 允许用数据缓冲里的哨兵值代替计数器 (Kog: 0.8 μs vs 7.6 μs)
  hw_tile_trigger: false          # Rubin 类硬件 tile 触发器; 有则 wait 不生成自旋
  backoff: s_sleep                # 64×N 周期，最多 ~8000 周期; s_setprio 0-3; s_memrealtime 1
00 MHz
costs:          # 阶段 0 标定后填入; 下表的公开数字是种子值
  t_local_ns: 116   t_cross_ns: 203   t_flag_cross_ns: 703   t_fence_ns: 115
  t_phase_switch_counter_ns: 7600   t_phase_switch_sentinel_ns: 850          # 全卡相位切换，两种
信号 lowering
  t_prologue_ns: null          # 每任务 prologue 爬坡，按资源类标定; Pass 3 / 7 的输入
  t_dev_ns: null   queue_contention: null

```

关键的一行是 `poll`：普通 load 命中非相干的 L1 (wave scope 是「Hit LRU」)，永远看不到别的 CU 的写；轮询事件必须用 `sc1` 位的 load 或原子读。Kog 为此写了内联汇编，因为 `__hip_atomic_load` 做不了 3-dword 读。这类事实只能以数据存在于表里，不能靠每个 tile 作者记得。

字段分两类，不允许混：`capabilities` 与 `visibility` 错了是错的代码；`costs` 错了只是慢。

7.3 可移植性矩阵

字段	MI250X (GFX90A)	MI300X (GFX942)	MI355X (GFX950)	RDNA4 (GFX12)	H100 (SM_90)	B200 (SM_90)
exec domain	2 GCD × 110 CU，各为一个 HIP 设备	8 XCD × 38 CU = 304（每 XCD 40 个，屏蔽 2 个）	8 XCD × 32 CU = 256（每 XCD 36 个，屏蔽 4 个）	WGP 阵列，单 domain	132 SM，单 domain（L2 两分区，远端命中 414 vs 258 周期）	148 SM die），单 domain（L2 两分区，远端命中 414 vs 258 周期）
L2 归属	8 MB / GCD	4 MB / XCD，跨 XCD 靠 wbl2 / inv + MTYPE 探测	4 MB / XCD，16 通道	统一	50 MB，两分区	126 MB 跨越约 100 个 XCD
共享缓存	无（3 级延迟）	256 MB Infinity Cache，内存侧，≈218 ns	256 MB，2 个 IOD	Infinity Cache（消费级）	—	—
LDS / smem 每 CU	64 KB	64 KB	160 KB，读带宽翻倍	64 KB / CU，WGP 模式 128 KB 共享，单 workgroup 上限 64 KB	228 KB	228 KB
wave	64	64	64	32（WMMA 仅 wave32）	32	32
cluster / DSMEM	无	无	无	gfx1250 起有 cluster scope	有，可移植上限 8，最大 16；DSMEM 181–213 周期 vs 全局 ~1110	有
异步拷贝到 LDS	无	仅 32 bit / lane	128 bit / lane，带转置读	gfx11 无；gfx1250 有 TDM	TMA / cp.async	TMA + 32 周期
多播归约写	无	无	无	无	multimem（NVLink SHARP）	multimem（NVLink SHARP）
浮点原子过互联	否（NOP 风险）	是（fp32 add、packed fp16 / bf16、fp64 add/min/max）	是	—	是	是
卡间互联	GCD 间 4 条 IF，200+200 GB/s；kernel 发起的 P2P 43–172 GB/s，延迟 8.7–18.2 μs	7 条 xGMI，64 GB/s 单向 / 链路，实测 ~48；点对点 mesh，无交换机	7 条 IF，聚合 > 1 TB/s	PCIe	NVLink 900 GB/s + NVSwitch	NVLink NVSwitch

字段	MI250X (GFX90A)	MI300X (GFX942)	MI355X (GFX950)	RDNA4 (GFX12)	H100 (SM_90)	B200 (RDNA4)
退避 / 计时	s_sleep	s_sleep (64×N 周期) · s_setprio · s_memrealtime	同左	同左	nanosleep (≤ 2t, 上限 1 ms) · griddepcontrol	同左
带宽	3.2 TB/s	5.3 TB/s	8 TB/s, 288 GB HBM3E	~0.6–0.9 TB/s	3.35 TB/s	8 TB/s

三个静默错误陷阱 (AMD 原子)

- 1. coarse-grained 内存上的 system-scope 原子会被降级为 device scope。跨卡事件张量必须用 fine-grained 分配。
- 2. PCIe 不支持某原子时，GPU 侧退化成 load-op-store，CPU 看到的不是原子序列。
- 3. MI200 的浮点原子不能过 Infinity Fabric，在 uncached 内存上可能直接 NOP。

结论：事件张量的分配器是 L4 的一部分，按 **scope** 选内存类型，并在编译期断言合法组合。这一条是我们把 scope 放进 IR 的直接原因。

7.4 代价表（决定性能）与标定

T_sync(edge) = T_local

= T_cross

= T_dev

T_push(edge) = T_sync(edge) + T_queue(N_contenders)

若 producer 与 consumer 同 domain

若跨 domain (经共享缓存 / 互联)

若跨 device (P2P / NVLink / xGMI)

系数不猜，靠阶段 0 的七个微基准填（附在第 15 节）。公开数字先作种子值：

测量	MI300X	H100 / B200	对设计
两 workgroup 原子往返，同 XCD vs 跨 XCD	116 ns vs 202 ns	B200 全卡约 512 原子 / 周期	跨 domain 至少 1.7x，占 Infinity Fabric
agent-scope release / acquire fence	115 / 137 ns	—	每个事件 arrive 一次 last-acquire 合并
跨 XCD 单向标志延迟	≈ 703 ns	DSMEM 181–213 周期 vs 全局 ≈ 1110 周期	MI300 个跨事件 ≈ 级，- 10 个 1% 的时间
持久化内核阶段切换：计数器 + fence vs 哨兵值 sc1 轮询	7.6–7.9 μs vs 0.80–0.93 μs (Kog, 256 CU)	—	全卡计数器昂贵；哨兵值 lower 选，哨兵抽象
XCD 感知的 program-id 重映射 (AMD 官方)	L2 miss ≈ 5M → 3.1M, +67 TFLOPS	—	纯放量有 10
L2 / Infinity Cache / HBM 延迟	81–108 / 218–258 / 342 ns	H100 L2 近 258 远 414 周期	事件记在哪，注的最短

fleet-mi300x 的实测与此一致：一层 MoE 把全局事件从 10 压到 3 的收益约 1 ms / token 量级（27 层），跨 XCD 事件的代价比 XCD 内高一个量级以上。

放置与调度 pass

七个 pass 按顺序跑，每个 pass 只读 L3 的表，输出都是数据，不改 IR 语义。

Pass 1 · tile 尺寸与资源类

输入 TileOp 的抽象资源需求与目标容量，输出每个算子的 tile 尺寸与 tile 数。64 KB 与 160 KB 的 LDS 会改变可行 tile 尺寸，所以同一算子在 gfx942 和 gfx950 上 tile 数不同，事件张量的 shape 随之变，这正是 shape 保持符号的好处。

这一步还要解决论文没提的**寄存器并集问题**：一个内核的 VGPR / LDS 占用取所有 tile 类型的最大值，最重的 tile 决定全内核的占用率。ETX 把 tile 按资源分成**资源类**；同一资源类的 tile 进同一个持久化内核实例。多个实例同时驻留（不同流、各占一部分 CU），共享同一张事件张量。这个「内核族」机制和多设备实例是同一套代码（见 L5）。在 fleet-mi300x 上我们测过 343 个寄存器的并集对带宽受限的 GEMV 没有损失，所以资源类是一个可选优化，不是必需项；代价模型决定要不要拆。

LDS 同理不按最坏情况分配，而是切成固定大小的页（Hazy 在 H100 上是 13 页 × 16 KB）。tile 体向页分配器申请和释放，下一个 tile 的权重装载可以在当前 tile 还在写回时开始。页数由资源类和 L3 的 `lds_kb` 算出：gfx942 4 页、gfx950 10 页、sm_90 13 页。

Pass 2 · 事件亲和性分片（chiplet 感知的核心）

问题形式：把任务 / 事件图切成 k 块（k = domain 数），最小化跨块边权之和，边权 = 扇入扇出数 × 该边的 `T_sync`。第一版用贪心加局部改进：

1. 以扇入扇出最大的算子为种子，把它和它的事件放进同一 domain（MoE 的 GroupGEMM 与 expert 事件；attention 的 head 与它的 merge）。
2. 连续 tile 坐标落同一 domain，再叠 `GROUP_SIZE_M` 式 swizzle 保 L2 局部性。
3. 跨 domain 边超预算时，把该事件提升到共享缓存作汇聚点，或把这条边改成静态长队列（不 push）。

输出：每个任务的 `domain_id`、每个事件的 scope 与存储类型（普通 VRAM / 共享缓存常驻 / 跨卡 fine-grained）。fleet-mi300x 的一层 MoE 里，这一步和 Pass 3 一起把全局事件从 10 压到 3，其余 24 个都是 XCD 局部的。

Pass 3 · 事件消除：以重算换同步（杠杆 1）

一个事件存在，是因为某个生产者的输出要被很多消费者用。如果生产者纯函数、它的输入已经被更早的事件同步过、而且它便宜，那就让每个消费者在自己的 prologue 里重算它，事件和生产者任务一起删。付出的是并行的冗余微秒，换回的是串行的同步微秒。

消除前 · 两个相位



冗余字节: $296 \times 4 \text{ KB} = 1.2 \text{ MB}$,
全部命中 L2, 不占 HBM

消除后 · 一个相位



fleet-mi300x: 这一类改写
把一层的全局事件 $10 \rightarrow 3$

以重算换事件。编译器里叫 rematerialization（用重算代替 spill），HPC 里叫 communication-avoiding；这里省的是同步。batch 1 时每个相位本来就是几百个 worker 并行做同一件小事，冗余是并行的，事件是串行的，所以冗余几乎总是赢。

变体	做法	删掉什么	冗余代价
prologue 重算	消费者各自重算小生产者（norm、RoPE、kv 后处理、q-absorb、router top-k）	生产者任务 + 事件	消费者数 $\times$ 生产者代价，并行
消费者侧折叠	不设 reduce 任务，每个消费者在 prologue 里把 k 个 partial 加起来	reduce 任务 + 事件	消费者数 $\times$ k 个 partial 的读取（fleet： $296 \times 8 \times 4 \text{ KB}$ ，走 L2 / Infinity Cache）
最后到达者计算	事件计数归零的那个生产者顺手做归约	reduce 任务，事件保留	归约变串行；fleet 测出每个折叠点约 $4 \mu\text{s}$ ，反而更慢，默认关

收益 = $T_{\text{sync}}(E) + T_{\text{drain}}(\text{生产者相位})$ # 同步往返 + 只有少数 worker 在跑的那段

代价 = $\text{max_consumer}(\text{bytes_P} / \text{BW_cache} + \text{compute_P})$ # 并行，取最大不取总和

+ [P 的输入不在 L2] $\times$ consumers $\times$ bytes_P / BW_HBM # 这时取总和，通常直接否决

改写条件 = P.pure $\wedge$ P 的 in_events $\subseteq$ C 已等过的事件 $\wedge$ P.out_to_in 能按 C 的坐标映射

$\wedge$ 收益 > 代价 $\wedge$ 逐 bit 要求下 P 的浮点求和顺序与参考一致

改写本身很机械：把 P 的 body 按坐标映射内联到 C 的 prologue，删 E，C 的 in_edges 换成 P 的 in_edges（映射后），然后重跑 L2 的无环和计数守恒校验。三条不能用的情况要写进日志：生产者要重读 HBM 里的权重（q-absorb 是边界案例，一层多读 2.1 MB，只因同 head 的 chunk 在同一 XCD 命中 L2 才划算）；partial 数超过 8 左右的大归约；消费者数太多且输入不在缓存里。日志打印被删掉的事件和换来的冗余字节，退化时能查到是哪次重算读爆了。

Pass 4 · 调度模式选择

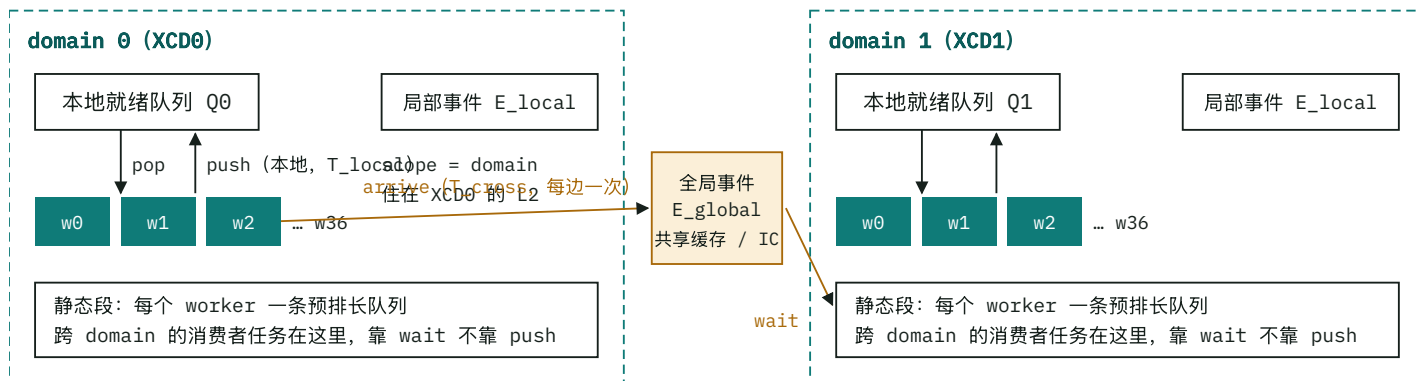
S_balance = 动态调度预计省下的 straggler 时间（由负载方差估计或历史 profile）
C_cross = 跨 domain push 次数 × T_cross（跨设备则 × T_dev）
C_queue = 队列争用代价（标定曲线，随竞争者数上升）

选 dynamic 当 S_balance > C_cross + C_queue 且无跨设备边
选 hybrid 当 domain 内有不规则负载、domain 间边多
否则 static

决策必须可解释，编译日志形如：

```
subgraph moe.group_gemm : hybrid (S_balance=+7.6%, intra-domain dynamic, 3 cross-domain edges → static)
subgraph attn.qkv_rope : static (S_balance≈0, 0 cross-domain edges)
subgraph tp.gemm_rs : static (cross-device edges=18, C_cross=2.1ms > S_balance=0.4ms)
```

Pass 5 · 队列分层（hybrid 的机制）

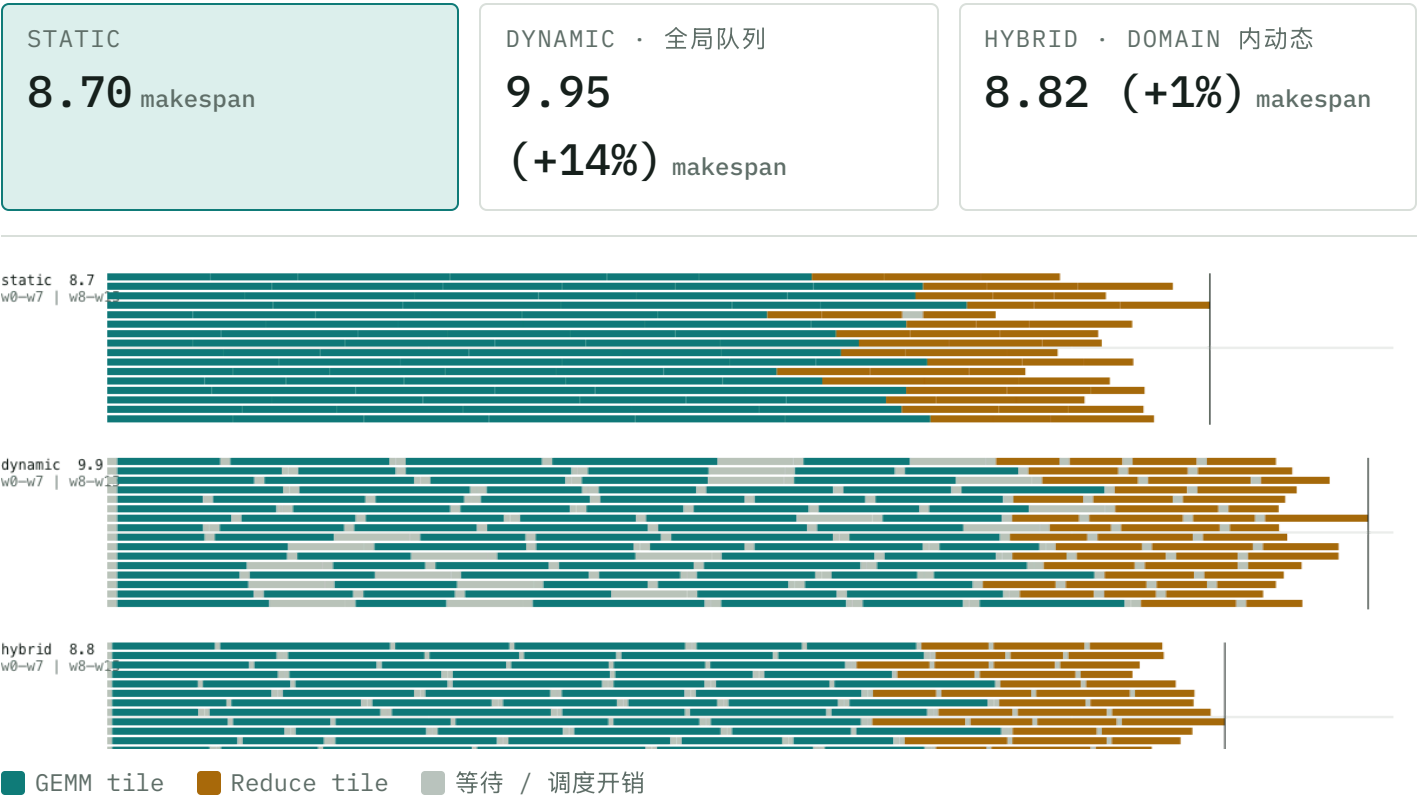


没有跨 domain 的 push。domain 内的不规则（MoE 路由）由本地队列吸收，domain 间的边数由 Pass 2 压到最少并静态排队。跨 device 时 E_global 住在 fine-grained 内存，形状完全相同：MI250X 的双 GCD 与 8 卡 TP 是同一条路。

hybrid 调度不是新机制，是同一套原语的不同组装。 domain 内用 push / pop，domain 间用 arrive / wait 加预排队列。全局队列只放跨 domain 任务，条目上限等于跨 domain 边数，这样中心化队列的争用就被结构压掉了。

交互 · 调度策略与拓扑代价

一个离散事件模拟：GEMM 类 tile（青）两两产生一个 Reduce 类 tile（琥珀），worker 分成两个 domain。拖动方差看 static 何时输给 dynamic，拖动跨 domain push 代价看 dynamic 何时反过来输给 hybrid。这是 Pass 4 公式的直观版，不是性能预测。



Pass 6 · 内存规划与事件分配

- 中间张量的生命周期与放置：domain 内消费的中间量优先放能被该 domain 的 L2 抓住的位置（fleet-mi300x：每层 1.2 MB 的压缩 KV 缓存整层留在 XCD 的 4 MB L2 里）。
- 事件张量按 scope 选内存类型（普通 / uncached / fine-grained），编译期断言与 L3 的 visibility 表一致。
- 静态路径把每个 worker 的预排队列物化到全局内存；描述符压成定长记录（subgraph_id, coord, task_type），不做指针追逐。

Pass 7 · 跨屏障权重预取（杠杆 3）

屏障挡住的是激活值。权重不依赖任何事件，静态可知，所以 worker 在自旋等事件的那段时间可以已经把下一相位自己那份权重流进 LDS 或寄存器，事件一到直接算。效果是每个屏障的固定延迟被下一相位的字节填满。

不重叠： [phase k 算][等事件 · 空转] [phase k+1 读权重][算]
重叠： [phase k 算][等事件 · 同时读 k+1 权重][算]

这个 pass 做三件事：

1. **静态段的下一任务可知**：worker 的预排表里下一个任务是确定的，编译器把它的权重切片地址算进当前任务的 epilogue；动态段用 early push 之后也能知道，只是晚一些。
2. **落点由能力表决定**：gfx942 没有 TMA，直接到 LDS 只有 32 bit / lane，预取进 VGPR / AGPR 或只做 L2 预热；gfx950 有 128 bit / lane 的直接到 LDS；Hopper / Blackwell 走 TMA 加 mbarrier。这是「能力决定做法」的典型例子，不能写成一段通用代码。
3. **与资源类、LDS 分页一起决策**：预取占寄存器和 LDS 页，会挤掉当前 tile 的占用。fleet-mi300x 单独加过一版 `--prefetch-next`，慢 4%，原因是寄存器压力和 L2 污染；Hazy 之所以能到 78% MBU，是分页和预取一起设计的。所以 Pass 1 的资源类要把「预取缓冲」算作 tile 的资源需求，代价模型按 `T_fixed(屏障)` - 预取造成的占用损失 决定开关。

预取和 Pass 3 是互补的：Pass 3 删掉能删的屏障，Pass 7 把删不掉的屏障的等待时间用掉。

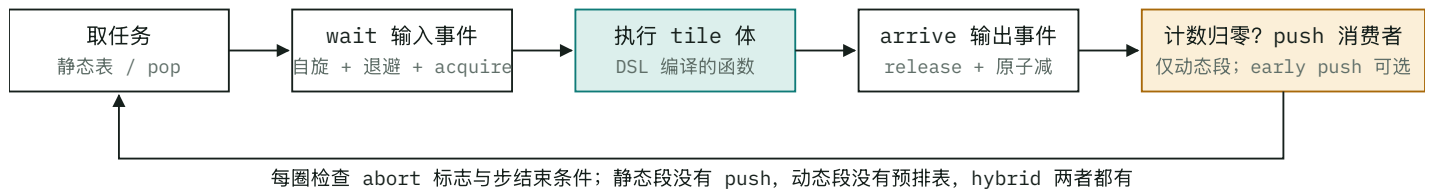
后端 lowering 与运行时 ABI

9.1 原语 × 架构 lowering 表

原语	NVIDIA SM_90 / SM_100	CDNA3 / CDNA4	CDNA2
arrive , domain scope	(无 domain 一级, 退化为 device)	s_waitcnt vmcnt(0) (L1 写穿) + device-scope 原子; 计数器落本 XCD 的 L2, 不发 fence (Fleet 与 fleet-mi300x 都验证过 0 个陈旧字)	同, GCD 内
arrive , device scope	fence.acq_rel.gpu + red.global.gpu.add	buffer_wbl2 sc1; s_waitcnt vmcnt(0) + 原子 (所有 RMW 转发到 Infinity Fabric) ; 生产者的 L2 回写不可省, 但同 XCD 的 N 个生产者可由最后到达者回写一次	同
arrive , system scope	red.global.sys.add	buffer_wbl2 sc0 sc1 + 原子; 事件必须在 fine-grained 内存 (hipExtMallocWithFlags), 否则静默降级	整数原子可以; 浮点原子不行
wait	ld.acquire.gpu 自旋 + nanosleep 退避; 有硬件 tile 触发器则不自旋	load sc1 (device) 或 load sc0 sc1 (system) 自旋 + s_sleep ; 返回后 buffer_inv sc0 (domain) / sc1 (device) / sc0 sc1 (system)。普通 load 会命中陈旧 L1, 是最常见的静默错误	同
组内低代价同步	cluster barrier + DSMEM (181–213 周期)	无, 退化为 domain 事件; gfx1250 起有 cluster scope	无
归约写 (通信融合)	multimem.ld_reduce	无对应物, 也无交换机: 对端 fine-grained HBM 上的 P2P store / 原子累加 (IPC handle 映射), 或 owner 侧 L2 驻留归约 (须实验)	无 (且浮点原子不能过 IF)
共驻保证	cudaLaunchCooperativeKernel	hipLaunchCooperativeKernel : CLR 按占用率算共驻上限, 超过返回 hipErrorCooperativeLaunchTooLarge (比死锁好) ; ROCm 自带的 grid.sync 就是 device-scope 原子 + s_sleep(1) 轮询	同
domain 探测	SM id 读 %smid , die 亲和可选测	启动时读 HW_ID 的 XCC_ID, 建 worker→domain 表; 不信 wgid % 8 (有公开记录测到 (k+4) mod 8)	设备 id 即 domain
LDS 页分配	smem 213 KB 切 13 页 × 16 KB (Hazy) ; 页由 mbarrier 守护	64 KB 切 4 页 × 16 KB; gfx950 160 KB 切 10 页; 页由 workgroup barrier 守护	同 gfx942
Triton host-DSL 模式的限制	原语齐全: atomics、 nanosleep 走 inline asm	AMD 后端有 buffer_atomic_rmw 、 buffer_load_to_local , 没有 s_sleep / setprio 钩子, 轮询退避走 inline asm; gfx942 无 async copy (gfx950 起有)	同 gfx942

表里每一格都来自 LLVM 的 gfx942 内存模型、CDNA3 ISA 或 ROCm 原子文档, 而不是推测; 新架构接入时这张表是第一份要填的东西。生成器只做一件事: 按事件的 scope 查表拼指令序列。

9.2 生成的内核骨架与 worker 状态机



运行时状态只有：事件缓冲（整型）· 队列（本地 + 全局）· 描述符表 · shape 标量 · worker→domain 表。没有任务图对象，没有解释器。

每个 worker 的循环。这是论文 §3.3 「极小运行时」的延续：调度逻辑编进内核，运行时只剩两块内存状态。static / dynamic / hybrid 的差异只在「取任务」与「push」两个格子里。

```
persistent_kernel(events, queues, desc_table, shape, dom_map, ctrl):
    w = worker_id(); d = dom_map[w] # 启动时探测，不假设映射公式
    sched = init_scheduler(d, mode_of_subgraph) # static 表 / 本地队列 / 两者
    while true:
        task = sched.next() # 静态表或 pop; 都空则看 ctrl
        if task.none: if ctrl.step_done(): break else: backoff(); continue
        wait_all(task.in_events) # 由 ETensor 依赖生成
        etx_tile_dispatch(task.type, ctx(task)) # switch 到各 DSL 的 tile 体
        for ev in task.out_events:
            if etx_arrive(ev, ev.scope) == 0 and ev.dynamic: sched.push(consumers_of(ev, task))
        if ctrl.abort: break
```

9.3 内核族：一个抽象覆盖多设备与多资源类

一次 Step 可以由多个内核实例执行，每个实例是一个 (device, resource_class) 对，共享同一张事件张量（同设备用普通 / uncached 内存，跨设备用 fine-grained）。三种场景是同一条路：

- MI250X：两个 GCD 是两个设备，一张卡启动两个实例，跨 GCD 事件是 system scope。
- 8 卡 TP：八个实例，跨卡事件 system scope，通信 tile 内联在各自实例里。
- 寄存器并集拆分：同一设备两个资源类（重 GEMM / 轻 GEMV），两个实例各占一部分 CU，事件 device scope。

调度器不知道实例，只知道 domain 树；实例只是树的一棵子树被交给了一个内核。

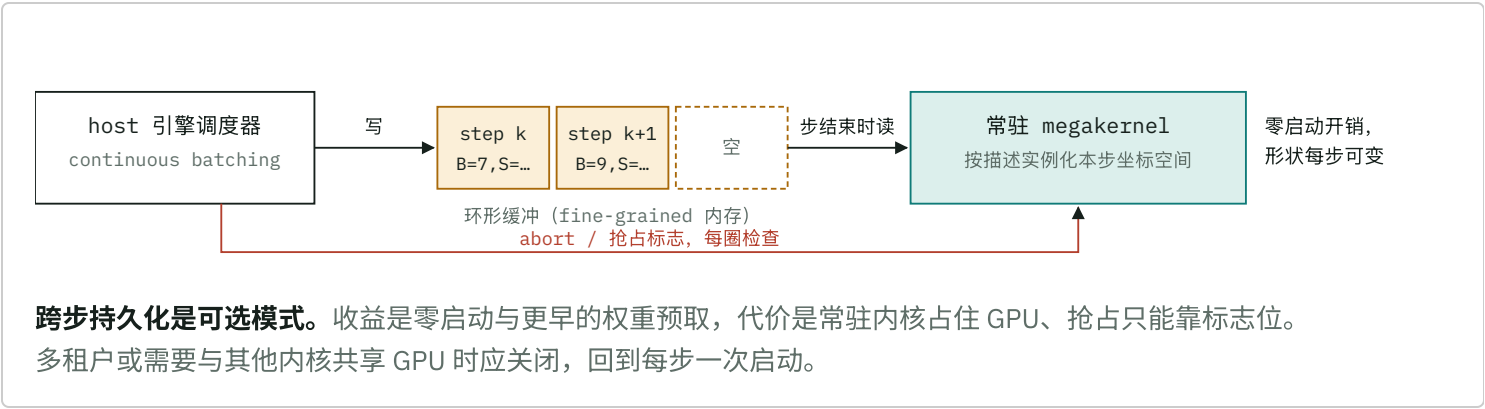
10.4 通信融合在没有 multimem 时怎么办

论文 1.40× 的 GEMM + Reduce-Scatter 建立在 multimem.ld_reduce (NVLink SHARP) 上，CDNA 没有对应指令。候选路径三条，都要单独实验，做不出正收益就在能力表里写 comm_fusion.reduce_scatter: unsupported，只做计算侧融合：

1. owner 侧归约：每个 rank 的 GEMM tile 把结果写到 owner rank 的 fine-grained 缓冲区，owner 的 RS tile 等事件后在本地 L2 归约。写是单向 P2P store，事件是跨卡 arrive。
2. P2P 浮点原子累加：MI300 支持浮点原子过 IF，MI200 不支持；精度上求和顺序不确定，需要 fp32 累加一次 bf16 舍入。
3. 借用 rocSHMEM / Iris 的一侧通信原语作为 tile 体，把 ETX 只当调度器用。

动态性的三个层次

层次	例子	ETX 机制	论文覆盖
shape 动态	continuous batching 的 B、S 变化；prefill / decode 混合	符号维度 + 每步 shape 标量；静态段用 shape 桶（而非「下一个更大」），桶内 worker 数按 tile 数缩放	覆盖
数据依赖动态	MoE 路由；speculative decoding 的接受长度；稀疏注意力的块选择	runtime 初始化的 wait_count + range / gather 边；domain 内动态队列吸收不均	覆盖 MoE；spec-decode 是同一机制的新实例（接受长度决定后续 tile 触发范围）
跨步动态	常驻内核跨多个解码步；请求随时加入 / 退出；多租户共置	host↔device 控制环：host 写步描述（shape 标量、指针表、事件初值）到环形缓冲，内核在步结束时取下一条；abort 标志；事件带 epoch 戳（mKernel 的做法），跨步不必清零，wait 比较的是 count == target(epoch)	未覆盖（每步一次启动）；MPK 在内核内做准入，Blink 把整个服务循环放进内核



三个层次共用一条原则：**编译产物是模板，不为任何具体值重编译**。变化的东西全部是启动参数或运行时张量。

负载覆盖矩阵

负载	动态性	预期调度	主要收益来源	风险
低 batch decode (dense)	shape	static	打破算子边界：Q 与 K 的 norm+RoPE 并行、GEMM 间流水、权重预取、去掉几百次启动	tile 质量不如手写库时收益被吃掉；论文 TP=4 上 CPU 侧开销反而是瓶颈
MoE decode / 小 batch prefill	数据依赖	hybrid	domain 内负载均衡 + GroupGEMM 两阶段流水 + 消除 wave 量化	跨 domain push 代价；路由极不均时本地队列也救不了
TP 通信融合 (GEMM+RS / AG+GEMM)	通信抖动	RS: dynamic (域内) ; AG: static	计算与通信重叠	CDNA 无 multimem ，需 10.4 的替代路径
大 batch prefill	shape	static 或不融合	收益有限，主要是不退化	回归测试项：论文称大 batch 不退化，我们要把它写成测试
speculative decoding	数据依赖 (接受长度)	hybrid	验证与下一步草稿的 tile 按接受长度触发，无 host 往返	树形草稿的依赖不是仿射的，需限制为链式或固定树
多模型 / 多租户共置	跨步	按 domain 隔离	domain 级隔离，互不拖累	队列与内存隔离；抢占只有标志位
非 LLM 的 tile DAG (扩散模型、视觉编码器)	多为静态	static	通用性证明	缺参考实现与基线

验证策略

正确性

- **协议模拟器**：host 上用 Python 模拟事件协议，对采样 shape 与随机路由跑整个任务图，检查无死锁、计数守恒、每个 tile 的输入在它执行前已被写。fleet-mi300x 已有原型（3 个 token 的事件协议模拟 + 12 个注入缺陷的变异测试）。
- **差分测试**：每个融合内核配一份未融合参考（逐算子提交、同 tile 代码 + 全局屏障），随机 shape、随机路由，逐元素比对。
- **scope 测试**：专门构造跨 XCD / 跨卡的事件，验证真的可见；能抓住系统域原子被静默降级。
- **原子能力断言**：对目标内存类型做一次原子读写回，不成立则编译期失败。
- **看门狗**：所有测试带超时，hang 视为失败，并 dump 事件张量（哪个事件计数没归零、谁没 arrive）。

性能

- 基线三档分开报：逐算子提交、同 tile 代码 + 全局屏障（隔离出「融合收益」、官方库（cuBLAS / RCCL、CK / hipBLASLt）。
- 每个 PR 跑固定 shape 集合，超阈值报警；报数一律带 shape、batch、并行度。
- 调度决策日志入库，事后能回答「为什么这个子图选了 static」。

可移植性回归

- 同一份 IR 在 gfx90a / gfx942 / gfx950 / sm_90 上跑同一个小模型：逐元素一致、不 hang。
- 决策日志必须打印能力位差异，证明决策真的读了机器模型，而不是碰巧一样。
- 能力表里每个 `discover` 字段（如 XCD 映射）在 CI 里跑探测并与 yaml 对账。

与 ETC / MPK / Hazy 的差异

维度	ETC（本论文）	MPK（MIRAGE）	HAZY NO-BUBBLES	ETX
抽象	Event Tensor（一等张量）	ttGraph：任务与事件的二分图，事件融合后每任务恰一个入事件一个出事件，描述符 352 B	指令流 + 每边一个计数器；共享内存分页	Event Tensor + scope / domain / init_kind / epoch
调度	static 或 dynamic，二选一	4 个专用调度 SM；worker 有 JIT / AOT 双队列；2026 路线图转向全静态	1B 静态每 SM 指令流；70B 全局原子计数器自取	static / dynamic / hybrid ，按子图由代价模型选
硬件	B200，NVLink	NVIDIA；AMD 由 Fleet 移植（MI350X，每 XCD 一个调度 workgroup，分层事件，比直接移植快 1.13–1.30×）	H100 / B200	CDNA2/3/4、RDNA、Hopper / Blackwell，能力表驱动
chiplet	无概念	Fleet 补上：四级任务（wave / CU / chiplet / device）+ XCD 内 L2 本地计数	无	exec-domain 树是一等概念；事件有 domain scope；调度器分层
动态 shape	符号 shape 模板	按 2 的幂 batch 特化多张图；MoE 用元张量半静态切分	1B 固定 batch；70B 靠全局队列吸收	符号 shape 模板 + shape 桶 + 本地队列
DSL	声明可接 Triton / CuteDSL，实现是 TVM DSL	自有超优化器 + 转译器，任务切分仍手工	ThunderKittens 手写	最小契约 + link / host-DSL 两种模式
内存类型	未展开	未强调	—	事件按 scope 分配，非法组合编译期报错
跨步	每步启动	内核内准入 / 驱逐（起始事件触发）	每步启动	可选常驻 + 控制环 + epoch 事件
tile 质量	编译器生成，承认不如 cuBLAS	编译器生成，寄存器按最大任务分配（占用率问题）	手写，78% HBM 带宽	允许内联手写 / 厂商 tile；资源类拆内核族；LDS 分页

还有两家值得单列。**Cohere（2026-09）**：host 预排静态波次 + 按算子的 drain 队列做工作窃取，固定 12-warp ABI，H100 上 30B-A3B bs=1 对 vLLM 1.58×，但 batch ≤ 8、无多卡。它证明 hybrid 在真实 MoE 服务上成立，ETX 的 domain 分层与它的算子分层正交。**Ada-MK（百度）**：把调度全部前移到编译期（PTX 级 DAG、共享内存页状态机），在没有 TMA 的 L20 上部署到广告推理；它是「静态段」做到极致的样子，也说明静态段值得单独优化而不是当作动态段的退化。

里程碑、风险、未验证清单

阶段	交付	完成定义
0 · 基线与标定	MI300X 上 MoE / dense 的逐算子参考 + 计时脚手架 + 七个同步代价微基准	未融合基线到手； <code>t_local / t_cross / t_dev</code> 与争用曲线入 yaml
1 · 单卡闭环	L2 IR + L5 codegen (link 模式, 手写 tile), static, 事件按 XCD 分片	MoE 层 $\geq 1.05\times$ 未融合基线；差分测试通过；协议模拟器接入 CI
1.5 · 关键路径	Pass 3 事件消除 + Pass 7 跨屏障预取 + 哨兵信号 lowering；每相位 trace 工具	MoE 层的全局事件数降到天然依赖点数 (3)；每层固定延迟占比从约 75% 降到 50% 以下；MBU 报表进 CI
2 · 动态与能力表	本地队列、hybrid、能力表 + 代价表、决策日志	MoE 上 hybrid $\geq$ static (对标论文 1.08 vs 1.04)；日志可解释
3 · 第二前端 + 第二硬件	Triton host-DSL 模式；同一 IR 跑 gfx950 或 gfx90a (双实例)	换前端 / 换硬件只改 yaml 与骨架模板, IR 与 pass 零改动
4 · 通信融合	GEMM + RS 在 MI300X 上的无 multimmem 路径	有正收益就报数, 没有就在能力表标不支持
5 · 跨步常驻	控制环、abort、shape 桶	continuous batching 下 TPOT 不劣于每步启动, 且抢占延迟有上界

风险与退路

风险	为什么	先测什么	失败时的退路
AMD 无 multimmem	论文的通信融合依赖它	10.4 的三条路径单点实验	标不支持, 只做计算侧融合
跨 XCD 同步代价高	4 MB 私有 L2, 跨域只有 Infinity Cache	阶段 0 标定	提高同域分片比例, 牺牲并行度
cluster / DSMEM 能力按目标变	LLVM 文档: 不支持时退化为 agent scope	逐目标实测写进 yaml	一律走全局原子
动态队列争用	论文自认中心队列有争用	争用曲线标定	domain 级队列 (已是默认)
tile 质量不如手写库	ETC 自己承认	同构对比	融合与 tile 解耦, 内联手写
静态调度的 shape 兜底粗	论文用「下一个更大 shape」	shape 长尾实验	shape 桶 + 桶内动态
事件内存类型写错	系统域原子有内存类型要求	scope 测试	编译期断言禁掉非法组合
持久化内核难调试	hang 不报错	看门狗 + 事件 dump 先于一切功能	没有退路, 这是必需品

阶段 0 的七个微基准

1. 同 XCD 内两个 workgroup 的原子计数往返延迟 (`t_local`)。
2. 相邻 XCD 与最远 XCD 的同一测量 (`t_cross` , 取两档)。
3. 事件张量放普通 VRAM、uncached、fine-grained 三种内存的差异。
4. 双卡 P2P 原子 + fence 的延迟与带宽 (`t_dev`)。
5. N 个 workgroup 抢同一队列的吞吐曲线, 扫 N (争用曲线)。
6. 静态队列长度与 straggler 时间的关系 (静态调度的适用边界)。
7. MoE 路由不均程度与动态调度收益的关系 (复现论文 1.08 vs 1.04 的条件)。

来源

- Jin, Hou, Wang, Lai et al. Event Tensor: A Unified Abstraction for Compiling Dynamic Megakernel. MLSys 2026. 本文所有「论文」数字均出自 v2。
- Cheng et al. Mirage Persistent Kernel: A Compiler and Runtime for Mega-Kernelizing Tensor Programs.
- Spector et al. Look Ma, No Bubbles! Designing a Low-Latency Megakernel for Llama-1B.
- Zheng et al. Triton-Distributed: Programming Overlapping Kernels on Distributed AI Systems with the Triton Compiler.
- Hou et al. Axe: A Simple Unified Layout Abstraction for Machine Learning Compilers (ETC 的 tile DSL)。
- AMD. Fleet: chiplet-aware megakernel on MI350X (MPK 的 ROCm 移植, 四级任务、分层事件、每 XCD 一个调度 workgroup) ; ROCm/fleet-chiplet-megakernel。
- Cohere. Megakernels (2026-09, H100, 静态波次 + 按算子 drain 队列, 30B-A3B bs=1 对 vLLM 1.58x)。
- Kog. Building a single-kernel latency-optimized LLM inference engine on MI300X (哨兵值轮询 0.8 μ s vs 计数器 7.6 μ s)。
- Ada-MK 2605.11581 · ForgeMegakernel 2609.12379 · mKernel 2609.13585 · DITRON 2605.02953 · Iris 2511.12500 (ROCm/iris)。
- Chips and Cheese. Testing AMD's giant MI300X (同 XCD / 跨 XCD 原子 116 / 202 ns) ; Hopper 微基准 (DSMEM 与 L2 分区延迟) ; NVIDIA Blackwell tuning guide; PTX ISA (multimem、nanosleep、griddepcontrol)。
- AMD. MI300 微架构; MI350 微架构; SPX / CPX 分区与 round-robin 派发; device-libs cg.cl (grid.sync 实现)。
- AMD. ROCm GPU architecture specs; GPU atomics operation support; MI300 / MI350 workload optimization (XCD-aware swizzle) ; MI300X partitioning; AMD Instinct MI300 CDNA3 ISA Reference Guide (2025-08)。
- LLVM. AMDGPU backend usage (gfx942 memory model: wbl2 / inv 序列、cluster launch) ; Clang AMDGPU builtins (s_sleep、s_setprio、s_memrealtime)。
- cklxx. fleet-mi300x: MI300X 上的手写 batch-1 MoE 解码 megakernel, 本文引用的跨 XCD 事件、fence、探测经验出自其 docs/STATUS.md。